ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Nguyen Phan Anh**
- **Chinese Name: 阮潘英**
- **Email: nguyenphananh.study@gmail.com**
- **Student ID: 115065426**
- **University: NTHU – National Tsing Hua University**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1: 594 documents

2: Because each category has its own vocabulary and patterns. If we don't split them out before applying algorithms and mine the whole at once, the most frequent patterns will be the ones shared by all categories. By isolating a category first, frequent pattern mining can find words that are characteristic of that domain (e.g., "doctor", "disease", "treatment" in sci.med). It also reduces the amount of data and the vocabulary the algorithm has to process, making mining faster.

AI used for this answer: No

If yes, how:

External references (links):

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer: 40% is almost half of the dataset. Dropping these rows could mean losing a significant amount of valuable data. It could also make the remaining data distribution uneven, leading to bias in the dataset. One alternative way is to train a classification model using documents with known category_name labels as the training set, and then use the trained model to predict the missing category_name labels for the unlabeled documents.

AI used for this answer: No

If yes, how:

External references (links):

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1: No, it's roughly the same but not exactly the same

2: A random sampling on an imbalanced dataset is still an imbalanced dataset, it could lead to biased results, and the sample may not represent minority categories as well as the original data set

AI used for this answer: No

If yes, how:

External references (links):

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1: The total number of feature increases, from 35482 to 35788

2: The sparsity decreases, as new words introduced, the new columns are non-zeroes, so the percentage of zeros in the matrix goes down

AI used for this answer: No

If yes, how:

External references (links):

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

1: The total size of the vocabulary decreases as the variance threshold increases

2: Words that have low frequency and only appear in 1-2 documents

3: Beacause these words carry little to none information to categorize. A word that appears in almost all documents cannot effectively distinguish one document from another.

Threshold	**Number of terms kept (keep == True)**
0.0	35482
0.01	5089
0.05	1491

AI used for this answer: No

If yes, how:

External references (links):

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?

3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1: Top-correlated words are domain-specific terminology for graphics

2: Pearson: Measures the strength of the linear relationship between the raw values of two variables

Spearman: The same idea, but computed on the ranks of the values rather than the values themselves

3: When its relationship with the topic is monotonic but not linear, or when it contains extreme values

AI used for this answer: Yes

If yes, how: I asked AI to explain to me what Pearson and Spearman measures is, and how to calculate them

External references (links):

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup: When minSup is 3, the number of patterns is 9547 and runtime is 3 seconds. When minSup is 9, the number of patterns is 710 and runtime is 0.11 seconds. So reducing minSup from 9 to 3 increases the runtime and significantly increases the number of patterns

Answer (2) — Top-K k vs MaxFPGrowth minSup: In Top-k, k fixes the number of patterns to return - only the top k patterns with the **highest support** are kept; While in minSup, all patterns with support ≥ minSup are kept

Answer (3) — Maximal vs standard frequent patterns: A maximal frequent pattern has no frequent superset, and all of its subsets are also frequent, so it could implicitly

represents smaller patterns that need not be listed. This removes redundancy and makes the result compact

AI used for this answer: Yes

If yes, how: I asked AI to explain to me the funamentals of Top-K FAE, MaxFPGrowth and maximal frequent patterns

External references (links):

https://www.geeksforgeeks.org/dsa/maximal-frequent-itemsets/

https://www.geeksforgeeks.org/machine-learning/frequent-pattern-growth-algorithm/

https://www.researchgate.net/publication/4285466_Mining_Top-Rank-K_Frequent_Patterns

---

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1: X_counts has approximately 35,788 dimensions. Humans can only visualize 2–3 dimensions, so we cannot directly plot a scatter plot. Furthermore, in high-dimensional spaces, distances between data points tend to become more similar (the curse of dimensionality), making distance-based clustering less meaningful

2: Dimensionality reduction algorithms such as PCA, t-SNE, and UMAP should be applied first

AI used for this answer: No

If yes, how:

External references (links):

https://en.wikipedia.org/wiki/Curse_of_dimensionality

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1: t-SNE/UMAP shows distinct, separated cluster islands for the 4 document categories. PCA shows heavy overlap around the center, making it difficult to distinguish the four groups.

2: PCA is a linear projection that selects directions with the **highest variance**, while text data is a sparse matrix, where variance may be dominated by a few words or extreme documents (like the outliers in the plot). The topic-related structure may lie on a nonlinear manifold that PCA cannot capture. t-SNE and UMAP preserve local neighborhood structures, allowing them to separate clusters more effectively

3: t-SNE prioritizes preserving local neighborhood relationships rather than global distances, so the distances between clusters and their sizes in the plot do not reflect the actual distances in the original space. The results also depend on other parameters such as perplexity, random seed,…

AI used for this answer: Yes

If yes, how: I asked AI to explain to me the funamentals of PCA, t-SNE and UMAP

External references (links):

https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/

https://www.geeksforgeeks.org/machine-learning/umap-uniform-manifold-approximation-and-projection/

https://www.geeksforgeeks.org/machine-learning/ml-t-distributed-stochastic-neighbor-embedding-t-sne-algorithm/

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?

2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1: 4 binary columns

2: It implicitly assumes that the categories have an ordinal relationship and meaningful numerical distances. For example, category 3 is considered "greater than" category 1, and the distance between 0 and 1 is assumed to be the same as the distance between 2 and 3. This is meaningless for four distinct topics

3: One-hot binarization assigns a separate column to each category, so there is no order. All pairs of categories are equally distant (under Euclidean distance), preventing the model from being biased by any artificial ordering.

AI used for this answer: No

If yes, how:

External references (links):
